```
threadsafe_list(threadsafe_list const& other)=delete;
threadsafe_list& operator=(threadsafe_list const& other)=delete;

void push_front(T const& value)
{
    std::unique_ptr<node> new_node(new node(value));    <-4
    std::lock_guard<std::mutex> lk(head.m);
    new_node->next=std::move(head.next);     <-5
    head.next=std::move(new_node);      <-6
}

template<typename Function>
void for_each(Function f)     <-7
{
    node* current=&head;
    std::unique_lock<std::mutex> lk(head.m);     <-8
    while(node* const next=current->next.get())       <-9
    {
        std::unique_lock<std::mutex> next_lk(next->m);    <-10
        lk.unlock();                <-11
        f(*next->data);     <-12
        current=next;
        lk=std::move(next_lk);     <-13
    }
}
```

The `threadsafe_list<>` from listing 6.13 is a singly linked list, where each entry is a `node` structure ❶. A default-constructed `node` is used for the `head` of the list, which starts with a `NULL next` pointer ❷. New nodes are added with the `push_front()` function; first a new node is constructed ❹, which allocates the stored data on the heap ❸, while leaving the `next` pointer as `NULL`. You then need to acquire the lock on the mutex for the `head` node in order to get the appropriate `next` value ❺ and insert the node at the front of the list by setting `head.next` to point to your new node ❻. So far, so good: you only need to lock one mutex in order to add a new item to the list, so there's no risk of deadlock. Also, the slow memory allocation happens outside the lock, so the lock is only protecting the update of a couple of pointer values that can't fail. On to the iterative functions.

First up, let's look at `for_each()` ❼. This operation takes a `Function` of some type to apply to each element in the list; in common with most standard library algorithms, it takes this function by value and will work with either a genuine function or an object of a type with a function call